

Laboratório — Semana 4

Desenvolvimento de Sistemas Corporativos

Refatorando a Task API: @Service, DTOs, Strategy e Factory

1. Contexto

Na Semana 3 vocês construíram uma API de Tarefas (Task API) com armazenamento em memória, seguindo o fluxo **Controller** → **Service** → **Repository**. Naquele momento, o Controller devolvia a própria Entity **Task** como resposta.

Hoje vamos evoluir essa mesma aplicação, sem trocar de projeto, introduzindo:

1. **DTOs** (**TaskRequestDTO** e **TaskResponseDTO**) para proteger a Entity;
2. O padrão **Strategy**, para calcular a prioridade de uma tarefa;
3. O padrão **Factory**, para criar notificações de conclusão de tarefa.

O armazenamento continua em memória (lista ou mapa) — a persistência real com banco de dados é o tema da Semana 5.

2. Objetivos de aprendizagem

Ao final deste laboratório, você deve ser capaz de:

- Criar DTOs com *Java Records* e realizar a conversão Entity ↔ DTO dentro do Service;
 - Implementar uma interface **PriorityStrategy** com múltiplas estratégias concretas;
 - Implementar uma **NotificationFactory** que decide qual tipo de notificação instanciar;
 - Explicar, com suas próprias palavras, por que cada peça está em sua respectiva camada.
-

3. Preparação

Continue usando o projeto criado na Semana 3 (Spring Initializr, dependência **Spring Web**, sem banco de dados ainda). Confirme que a estrutura de pacotes está semelhante a esta:

```
src/main/java/br/edu/ifrn/taskapi/  
├── controller/  
│   └── TaskController.java  
├── service/  
│   └── TaskService.java  
├── repository/  
│   └── TaskRepository.java  
└── model/  
    └── Task.java
```

Vamos adicionar os pacotes `dto`, `strategy` e `factory`.

4. Parte A — Identificando o vazamento de Entity (10 min)

Antes de codificar, observe estes três trechos de resposta JSON que a API da Semana 3 devolve atualmente:

Trecho 1

```
{ "id": 3, "titulo": "Revisar contrato", "criadoPor": { "id": 12, "senhaHash": "8f14e45..." } }
```

Trecho 2

```
{ "id": 7, "titulo": "Migrar servidor", "auditoria": { "ultimaModificacaoInterna": "2026-09-09T02:11:00" } }
```

Trecho 3

```
{ "id": 9, "titulo": "Enviar relatório", "prazo": null, "versaoOtimista": 4 }
```

Em dupla, respondam por escrito:

1. Em cada trecho, qual campo **não deveria** estar visível para quem consome a API?
2. Que problema prático cada exposição indevida poderia causar (segurança, acoplamento, confusão do consumidor da API)?
3. Proponham os campos de um `TaskResponseDTO` que resolveria os três casos ao mesmo tempo.

Guardem a resposta — ela será comparada com o DTO que vocês implementarão na Parte B.

5. Parte B — Criando os DTOs

5.1. Crie o pacote `dto` com os dois records

```
package br.edu.ifrn.taskapi.dto;

import java.time.LocalDate;

public record TaskRequestDTO(
    String titulo,
    String descricao,
```

```
        LocalDate prazo  
    ) {}
```

```
package br.edu.ifrn.taskapi.dto;  
  
public record TaskResponseDTO(  
    Long id,  
    String titulo,  
    boolean concluida,  
    String prioridade  
) {}
```

5.2. Ajuste o `TaskService` para converter DTO ↔ Entity

```
@Service  
public class TaskService {  
  
    private final TaskRepository repository;  
  
    public TaskService(TaskRepository repository) {  
        this.repository = repository;  
    }  
  
    public TaskResponseDTO criar(TaskRequestDTO dto) {  
        Task tarefa = new Task(dto.titulo(), dto.descricao(), dto.prazo());  
        Task salva = repository.salvar(tarefa);  
        return toResponseDTO(salva);  
    }  
  
    public List<TaskResponseDTO> listarTodas() {  
        return repository.listarTodas().stream()  
            .map(this::toResponseDTO)  
            .toList();  
    }  
  
    private TaskResponseDTO toResponseDTO(Task tarefa) {  
        return new TaskResponseDTO(  
            tarefa.getId(),  
            tarefa.getTitulo(),  
            tarefa.isConcluida(),  
            tarefa.getPrioridade()  
        );  
    }  
}
```

5.3. Ajuste o `TaskController` para usar os DTOs

```
@RestController
@RequestMapping("/tarefas")
public class TaskController {

    private final TaskService service;

    public TaskController(TaskService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<TaskResponseDTO> criar(@RequestBody
TaskRequestDTO dto) {
        TaskResponseDTO criada = service.criar(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(criada);
    }

    @GetMapping
    public List<TaskResponseDTO> listar() {
        return service.listarTodas();
    }
}
```

Checkpoint: teste com o Postman ou Insomnia um `POST /tarefas` enviando `{ "titulo": "...", "descricao": "...", "prazo": "2026-10-01" }` e confirme que a resposta contém **apenas** os campos do `TaskResponseDTO` — nada de campos internos.

6. Parte C — Implementando o padrão Strategy

6.1. Crie a interface no pacote `strategy`

```
package br.edu.ifrn.taskapi.strategy;

public interface PriorityStrategy {
    String calcularPrioridade(Task tarefa);
}
```

6.2. Implemente as três estratégias

```
public class UrgentPriorityStrategy implements PriorityStrategy {
    public String calcularPrioridade(Task tarefa) {
        return "URGENTE";
    }
}

public class NormalPriorityStrategy implements PriorityStrategy {
```

```
        public String calcularPrioridade(Task tarefa) {
            return "NORMAL";
        }
    }

    public class LowPriorityStrategy implements PriorityStrategy {
        public String calcularPrioridade(Task tarefa) {
            return "BAIXA";
        }
    }
}
```

6.3. Faça o `TaskService` escolher a estratégia certa

Adicione um método privado que decide qual `PriorityStrategy` usar de acordo com o prazo, e aplique o resultado ao criar a tarefa:

```
private PriorityStrategy escolherEstrategia(LocalDate prazo) {
    if (prazo == null) {
        return new LowPriorityStrategy();
    }
    long dias = ChronoUnit.DAYS.between(LocalDate.now(), prazo);
    if (dias <= 1) {
        return new UrgentPriorityStrategy();
    } else if (dias <= 7) {
        return new NormalPriorityStrategy();
    }
    return new LowPriorityStrategy();
}
```

Use esse método dentro de `criar(TaskRequestDTO dto)` para definir `tarefa.setPrioridade(...)` antes de salvar.

Reflexão a registrar no relatório: o que mudaria no `TaskService` se amanhã surgisse uma quarta regra, por exemplo "prioridade máxima para tarefas de clientes VIP"? Vocês precisariam alterar o método `escolherEstrategia`, mas **não** as estratégias já existentes — por quê isso é uma vantagem?

7. Parte D — Implementando o padrão Factory

7.1. Crie a interface `Notification` e as três implementações

```
package br.edu.ifrn.taskapi.factory;

public interface Notification {
    void enviar(String mensagem);
}
```

```
public class EmailNotification implements Notification {
    public void enviar(String mensagem) {
        System.out.println("[E-MAIL] Enviando: " + mensagem);
    }
}

public class SmsNotification implements Notification {
    public void enviar(String mensagem) {
        System.out.println("[SMS] Enviando: " + mensagem);
    }
}

public class PushNotification implements Notification {
    public void enviar(String mensagem) {
        System.out.println("[PUSH] Enviando: " + mensagem);
    }
}
```

7.2. Crie a `NotificationFactory`

```
public class NotificationFactory {

    public Notification criar(String tipo) {
        return switch (tipo.toUpperCase()) {
            case "EMAIL" -> new EmailNotification();
            case "SMS" -> new SmsNotification();
            case "PUSH" -> new PushNotification();
            default -> throw new IllegalArgumentException(
                "Tipo de notificação desconhecido: " + tipo);
        };
    }
}
```

7.3. Use a fábrica ao concluir uma tarefa

No `TaskService`, adicione um método `concluir(Long id, String tipoNotificacao)` que:

1. Busca a tarefa no repositório;
2. Marca a tarefa como concluída;
3. Usa a `NotificationFactory` para criar a notificação apropriada e chama `enviar(...)` com uma mensagem informando a conclusão.

Exponha esse método em um novo endpoint no Controller, por exemplo:

```
@PatchMapping("/{id}/concluir")
public ResponseEntity<TaskResponseDTO> concluir(
    @PathVariable Long id,
```

```
@RequestParam String tipoNotificacao) {  
    return ResponseEntity.ok(service.concluir(id, tipoNotificacao));  
}
```

Checkpoint: teste `PATCH /tarefas/1/concluir?tipoNotificacao=SMS` e confirme, pelo console, que a notificação correta foi "enviada".

8. Desafio extra (opcional)

Para quem terminar antes do fim da aula: implemente uma quarta estratégia, `VipPriorityStrategy`, que sempre retorna `"URGENTE"` independentemente do prazo, e ajuste `escolherEstrategia` para usá-la quando a tarefa tiver um campo booleano `clienteVip = true`. Não altere nenhuma das estratégias já existentes — se precisar alterá-las, revise o design.

9. Entrega

Ao final da aula, envie por meio do canal indicado pelo professor:

- O código-fonte do projeto atualizado (pacotes `dto`, `strategy` e `factory` incluídos);
- Uma captura de tela do teste do endpoint `POST /tarefas` mostrando a resposta em formato DTO;
- A resposta escrita da Parte A (identificação do vazamento de Entity) e da reflexão da Parte C.

10. Critérios de avaliação desta prática

Critério	Peso
DTOs implementados corretamente (records, sem vazamento de Entity)	30%
Strategy implementado com as três variações funcionando	30%
Factory implementada e integrada ao fluxo de conclusão	25%
Qualidade das respostas escritas (Parte A e reflexão da Parte C)	15%